UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Carrera de Ingeniería de Software

Fichas bibliográficas

Actividad formativa individual previa a TA-IND-01

Erwin Daniel Bueno Troya
Asignatura: Sistemas Distribuidos

7 de noviembre de 2026

1. Decomposition of Monolith Applications Into Microservices Architectures: A Systematic Review

Referencia (IEEE)
[1]

DOI verificado
https://doi.org/10.1109/TSE.2023.3287297

Resumen
Muchas empresas quieren pasar sus aplicaciones monolíticas a microservicios de forma automática, pero no había un mapa claro de cómo hacerlo. Abgaz y su equipo aplicaron un protocolo de revisión sistemática y siguieron las referencias (snowballing). Empezaron con 5022 resultados y al final se quedaron con 35 artículos. Con eso armaron el marco M2MDF, que divide la descomposición en fases, y revisaron qué herramientas, métricas y conjuntos de datos se usan para comprobarla. Concluyen que el tema sigue en una etapa temprana: casi no hay métodos que mezclen datos estáticos, dinámicos y evolutivos, y tampoco hay métricas, conjuntos de datos ni líneas base estándar.

Dato concreto
El M2MDF tiene seis fases: recolección de entradas, análisis del monolito, identificación, optimización, evaluación y despliegue de microservicios. Sección IV-B, p. 4222.

Nota personal
Antes creía que migrar a microservicios era básicamente partir el código en varios servicios. Esta revisión me hizo ver que es un proceso por fases y que cada fase tiene sus propios problemas. En TA-IND-01 entra en la sección de Arquitectura del sistema, cuando explique cómo el sistema que analizo llegó a su arquitectura distribuida actual.

2. A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges

Referencia (IEEE)
[2]

DOI verificado
https://doi.org/10.1109/ACCESS.2023.3305687

Resumen
Sobre cómo migrar a microservicios hay bastante escrito, pero Velepucha y Flores notan que casi nadie habla de los principios que conviene seguir al construir esa arquitectura. El artículo es una revisión de literatura que parte de las ideas básicas de la programación orientada a objetos y las relaciona con las arquitecturas monolítica y de microservicios. Lo que aporta es una lista de patrones de microservicios y una comparación entre lo que proponen Martin Fowler y Sam Newman y la idea de ocultamiento de información de David Parnas. Al final resume las ventajas y desventajas de las dos arquitecturas y hacia dónde va el tema hoy.

Dato concreto
En la conclusión, los autores recomiendan que los microservicios se diseñen a partir del dominio del negocio (DDD), que se puedan reemplazar con facilidad, que oculten cómo están hechos por dentro y que cada uno tenga una sola responsabilidad de negocio. También piden automatizar la compilación y el despliegue, y que los microservicios guarden logs para poder vigilar su funcionamiento. Sección VI.

Nota personal
Gracias a esta fuente entendí que los microservicios salen de ideas viejas sobre modularidad, como las de Parnas, y no de algo totalmente nuevo. En TA-IND-01 la cito en la Postura crítica, porque su Tabla 4 de ventajas y desventajas me sirve para argumentar las fortalezas y limitaciones del diseño. Lo que dice sobre los logs también lo puedo llevar a la sección de Tolerancia a fallos, cuando explique cómo se da cuenta el sistema de que algo falló.

3. Evolution of Microservices Identification in Monolith Decomposition: A Systematic Review

Referencia (IEEE)
[3]

DOI verificado
https://doi.org/10.1109/ACCESS.2024.3365079

Resumen
Para pasar de un monolito a microservicios primero hay que identificar bien cada microservicio, y para Oumoussa y Saidi ese tema todavía está en evolución. Buscaron en bibliotecas digitales y siguieron las referencias de los artículos, con tres preguntas como guía: cómo ha cambiado el tema con los años, qué técnicas y herramientas son las más avanzadas y qué obstáculos quedan por delante. Encontraron que todavía faltan herramientas más automáticas y precisas para identificar servicios, y pruebas de evaluación estandarizadas para comparar resultados. También dicen que hace falta entender mejor el lado humano de una migración que sale bien.

Dato concreto
Seleccionaron 168 estudios: 35 salieron del primer filtro y otros 133 los añadieron después de revisar las referencias de esos artículos. Sección III-C.

Nota personal
Lo que más me quedó de esta fuente es que lo difícil de una migración es decidir dónde termina un servicio y dónde empieza el otro. Me sirve para la sección de Arquitectura del sistema, al describir los componentes del sistema que elegí y revisar si la forma en que está dividido tiene sentido.

4. Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation

Referencia (IEEE)
[4]

DOI verificado
https://doi.org/10.1109/ACCESS.2022.3152803

Resumen
Muchas empresas pequeñas pasan a microservicios esperando lograr lo mismo que Netflix o Amazon. Los autores dicen que en sistemas sin miles de usuarios a la vez, que pueden crecer con una máquina más potente, eso casi no se ha estudiado. Para probarlo programaron una aplicación web en cuatro versiones (monolito y microservicios, en Java y en C#.NET) y la probaron en local, Azure Spring Cloud y Azure App Service, midiendo con Apache JMeter. En una sola máquina el monolito rindió mejor. En Azure convenía más usar máquinas más potentes que agregar instancias, y pasado cierto número de instancias el rendimiento bajaba.

Dato concreto
Los autores concluyen que una arquitectura monolítica parece ser una mejor opción para sistemas simples y pequeños que no tienen que atender a muchos usuarios al mismo tiempo. Sección VIII, p. 20372.

Nota personal
Esta fue la fuente que más me hizo dudar de que los microservicios siempre sean mejores, porque los autores midieron en vez de opinar. Pienso apoyarme en ella para la Postura crítica, donde tengo que decir en qué casos el sistema no sería la mejor opción. Su conclusión sobre los sistemas pequeños con pocos usuarios me da justo ese tipo de argumento con respaldo.

5. Understanding and Addressing Quality Attributes of Microservices Architecture: A Systematic Literature Review

Referencia (IEEE)
[5]

DOI verificado
https://doi.org/10.1016/j.infsof.2020.106449

Resumen
Se ha trabajado bastante en mejorar la calidad de los sistemas con microservicios, pero faltaba ordenar lo que se sabe sobre sus atributos de calidad. Con una revisión de 72 estudios, el equipo

de Li encontró los seis atributos que más preocupan: escalabilidad, rendimiento, disponibilidad, monitorizabilidad, seguridad y facilidad de prueba. También reunieron 19 tácticas de arquitectura para atenderlos. Para ellos, la escalabilidad, que casi siempre se presenta como la gran ventaja de los microservicios, sigue dando problemas cuando hay que equilibrarla con el rendimiento. Además piden prestarle más atención a la mantenibilidad.

Dato concreto
Las 19 tácticas se reparten así: dos para escalabilidad, cuatro para rendimiento, cuatro para disponibilidad, cuatro para monitorizabilidad, tres para seguridad y dos para facilidad de prueba. Resumen (Abstract), art. 106449.

Nota personal
Con esta fuente dejé de hablar de «calidad» en general. Ahora puedo revisar por separado cosas como la disponibilidad o qué tan fácil es vigilar el sistema. Sus tácticas de disponibilidad las llevo a la sección de Tolerancia a fallos de TA-IND-01, y lo que dice sobre escalabilidad frente a rendimiento en la Postura crítica, donde tengo que discutir qué se gana y qué se pierde con el diseño.

Declaración de uso de IA

Usé Claude Opus 5.5 para localizar fuentes académicas sobre microservicios y comprobar que cada DOI lleva al artículo correcto. Un ejemplo de prompt que usé: «Busca artículos de revistas indexadas publicados entre 2021 y 2026 sobre la migración de monolitos a microservicios, con DOI verificable». También me apoyé en ella para corregir errores de gramática. El esqueleto del archivo .tex, es decir, la estructura de las fichas y su conexión con el .bib, lo armé con su ayuda. Cada dato de las fichas lo verifiqué abriendo los artículos originales en IEEE Xplore y ScienceDirect. El contenido final es mi responsabilidad.

Referencias

[1] Y. Abgaz et al., «Decomposition of Monolith Applications Into Microservices Architectures: A Systematic Review,» *IEEE Transactions on Software Engineering*, vol. 49, n.º 8, págs. 4213-4242, 2023. DOI: 10.1109/TSE.2023.3287297

[2] V. Velepucha y P. Flores, «A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges,» *IEEE Access*, vol. 11, págs. 88339-88358, 2023. DOI: 10.1109/ACCESS.2023.3305687

[3] I. Oumoussa y R. Saidi, «Evolution of Microservices Identification in Monolith Decomposition: A Systematic Review,» *IEEE Access*, vol. 12, págs. 23389-23405, 2024. DOI: 10.1109/ACCESS.2024.3365079

[4] G. Blinowski, A. Ojdowska y A. Przybyłek, «Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,» *IEEE Access*, vol. 10, págs. 20357-20374, 2022. DOI: 10.1109/ACCESS.2022.3152803

[5] S. Li et al., «Understanding and Addressing Quality Attributes of Microservices Architecture: A Systematic Literature Review,» *Information and Software Technology*, vol. 131, 2021, **artno** 106449. DOI: 10.1016/j.infsof.2020.106449